



Documentación Técnica

Zgaming

Zgaming

1. Descripción general

Zgaming es una aplicación web de catálogo de videojuegos para explorar, buscar y guardar juegos favoritos. No tiene sistema de cuentas ni login: cualquiera puede navegar libremente, y la lista de guardados (**Wishlist**, hasta 20 juegos) vive únicamente en el **localStorage** del navegador de cada usuario.

Permite buscar por texto libre y filtrar por género, plataforma o tienda desde la página de búsqueda, o navegar directamente el catálogo de géneros/plataformas/tiendas desde secciones dedicadas y desde el mega menú del navbar (con vista previa en cards y acceso rápido).

2. Stack tecnológico

Categoría	Tecnología	Notas
Framework	Next.js 16	App Router, export estático (output: 'export')
Lenguaje	TypeScript 5	—
UI Library	React 19	—
Componentes	PrimeReact 10 + PrimiIcons	—
Estilos	Tailwind CSS v4	—
Botones animados	react-ladda-button	—
Backend	Vercel Functions (@vercel/node) + MongoDB	Driver oficial de MongoDB
Persistencia de usuario	localStorage	Wishlist, aviso inicial — sin base de datos de usuarios
Linting	ESLint 9	—

3. Arquitectura

El frontend es una app Next.js exportada como sitio estático (**output: 'export'**). Los datos de juegos **no** se piden a un backend externo: las funciones serverless de Vercel bajo **api/games** se conectan **directamente a MongoDB** (colecciones **database**, **genres**, **platforms**, **parent-platforms** y **stores**), replicando en TypeScript la lógica de filtrado/orden que antes vivía en un servicio .NET (**CesarSobGamesService**).

Así, en producción, tanto el sitio estático como las funciones se despliegan juntos en Vercel sin depender de un servidor propio siempre encendido.

```

% % % api/
% % % games/index.ts
% % % _lib/
%
% % % app/
% % % page.tsx
% % % layout.tsx
% % % globals.css
% % % game/
% % % search/
% % % searchGenres/
% % % searchPlatforms/
% % % searchStores/
% % % wishlist/
% % % legal/
%
% % % components/
% % % ClientProviders.tsx
% % % ToastProvider.tsx
% % % WelcomeDialog.tsx
% % % game-carousel/
% % % hero-header/
%
% % % features/
% % % home/ detailed-game/ search/ search-genres/
% % % search-platforms/ search-stores/ wishlist/
%
% % % services/
% % % config.ts
% % % GameService.ts
% % % wishlist.ts
% % % InfoService.ts
%
% % % models/
% % % shared/
% % % public/

```

Vercel Functions (junto al sitio estático)
 Endpoint único: lista, búsqueda, detalle,
 géneros/plataformas/tiendas
 Conexión a Mongo y lógica de filtrado/orden

Rutas y páginas (Next.js App Router)
 Página principal (Home)
 Layout global
 Estilos globales y variables de color
 Detalle de un juego (?slug=)
 Búsqueda general con filtros
 Catálogo de géneros
 Catálogo de plataformas
 Catálogo de tiendas
 Juegos guardados (localStorage) con filtros
 Política de privacidad y condiciones de uso

Componentes reutilizables
 Proveedores globales de cliente
 Notificaciones globales (límite Wishlist)
 Aviso inicial de la web
 Carrusel y card de juego (botón Wishlist)
 Cabecera hero de la home

Lógica de negocio por sección

Servicios y llamadas a la API
 URL base por entorno
 Cliente del endpoint /api/games
 Lectura/escritura de la Wishlist
 Datos auxiliares

Tipos e interfaces TypeScript
 Navbar (mega menú), Footer, enlaces sociales
 Recursos estáticos

No existe `vercel.json`: Vercel detecta el framework Next.js automáticamente (incluido `output: 'export'`) y despliega el export estático junto a las funciones de `api/` sin configuración manual de `build/output`.

4. Rutas de la aplicación (App Router)

Ruta	Página	Descripción
/	app/page.tsx	Home: hero, carruseles y catálogo destacado
/game?slug=	app/game/	Detalle de un juego
/search	app/search/	Búsqueda general con filtros (género, plataforma, tienda)
/searchGenres	app/searchGenres/	Catálogo de géneros
/searchPlatforms	app/searchPlatforms/	Catálogo de plataformas
/searchStores	app/searchStores/	Catálogo de tiendas
/wishlist	app/wishlist/	Juegos guardados (localStorage) con filtros

Ruta	Página	Descripción
/legal	app/legal/	Política de privacidad y condiciones de uso

5. Capa de datos (API)

5.1 Vercel Functions (api/)

- `api/games/index.ts`: **endpoint único** que resuelve listado, búsqueda, detalle y catálogos de géneros/plataformas/tiendas mediante parámetros de query.
- `api/_lib/`: conexión a MongoDB y lógica compartida de filtrado/orden (reimplementación en TypeScript de la lógica que antes vivía en el servicio .NET `CesarSobGamesService`).
- Colecciones de MongoDB usadas: `database`, `genres`, `platforms`, `parent-platoforms`, `stores`.

5.2 Cliente HTTP (services/GameService.ts)

Encapsula las llamadas del frontend al endpoint `/api/games`.

5.3 services/config.ts

Resuelve la URL base según el entorno, para endpoints ajenos a `/api/games`.

6. Persistencia de usuario (Wishlist)

- `services/wishlist.ts`: lectura/escritura de la Wishlist en `localStorage`.
- Límite: **20 juegos** guardados. Al superarlo se dispara una notificación mediante `ToastProvider`.
- No requiere cuenta ni servidor: todo el estado vive en el navegador del usuario.
- `WelcomeDialog.tsx`: aviso inicial (mostrado una vez) que explica el funcionamiento de la web; su estado de "leído" también se guarda en `localStorage`.

7. Componentes y features clave

Elemento	Responsabilidad
<code>components/ClientProviders.tsx</code>	Agrupar proveedores globales de cliente (tema, toast, aviso inicial)
<code>components/ToastProvider.tsx</code>	Sistema de notificaciones globales (ej. límite de Wishlist alcanzado)
<code>components/game-carousel/</code>	Carrusel y card de juego, con botón de añadir/quitar de Wishlist
<code>components/hero-header/</code>	Cabecera hero de la página principal
<code>shared/</code>	Navbar con mega menú, Footer y enlaces sociales, compartidos en toda la app
<code>features/*</code>	Lógica de negocio y composición de UI por cada sección

8. Variables de entorno

Variable	Descripción
MONGODB_URI	Cadena de conexión a la base de datos MongoDB
MONGODB_DB	Nombre de la base de datos a usar

Se configuran copiando `.env.local.example` a `.env.local` en desarrollo, y como variables de entorno del proyecto en el dashboard de Vercel en producción.

9. Cómo iniciar el proyecto

9.1 Requisitos previos

- Node.js v18 o superior
- npm (incluido con Node.js)
- Una base de datos MongoDB (Atlas u otra) con las colecciones del catálogo

9.2 Instalación y arranque en desarrollo

Como las funciones de `api/` son Vercel Functions (no rutas de Next.js), `npm run dev` (que solo levanta `next dev`) **no las sirve**. Para probar el sitio completo, incluida la conexión a Mongo, se usa la CLI de Vercel:

```
npm install
npx vercel dev
```

La app queda disponible en `http://localhost:3000`. Si solo se necesita trabajar en UI que no depende de `/api/games`, `npm run dev` es suficiente y más rápido.

9.3 Scripts disponibles

Comando	Descripción
npm run dev	Modo desarrollo (<code>next dev --webpack</code>) — no sirve <code>api/</code>
npm run dev:turbo	Modo desarrollo con Turbopack (más rápido)
npm run build	Compila para producción
npm start	Inicia en modo producción (requiere build previo)
npm run lint	Ejecuta ESLint

10. Despliegue

Proyecto pensado para desplegarse en **Vercel**:

- Vercel detecta el framework Next.js automáticamente, incluido `output: 'export'`.
- Despliega el export estático junto con las funciones serverless de `api/`, sin necesidad de `vercel.json`.
- Requiere configurar `MONGODB_URI` y `MONGODB_DB` en las variables de entorno del proyecto en Vercel.

11. Convenciones de código

- Next.js App Router con estructura de carpetas por ruta (`app/`).
- Lógica de negocio y composición de UI separada por sección en `features/`, manteniendo `app/` centrado en el enrutado.
- Tipos e interfaces TypeScript centralizados en `models/`.
- Elementos compartidos entre rutas (Navbar, Footer, enlaces sociales) en `shared/`.
- Persistencia de cliente (Wishlist, aviso inicial) implementada con `localStorage`, sin gestor de estado externo.

12. Limitaciones conocidas

- No existe autenticación ni backend de usuarios: la Wishlist es local al navegador/dispositivo y se pierde al borrar `localStorage`.
- Sin sincronización entre dispositivos.
- Límite fijo de 20 juegos en la Wishlist, sin posibilidad de configurarlo desde la interfaz.
- Las funciones de `api/` no se sirven con `npm run dev`; es obligatorio usar `vercel dev` para probar la integración completa con MongoDB en local.